

Tutorial 01

Tutorial 01 Notes

Overview

Course Info

Pointers Revision

Intro to Malloc

Linked List Revision

Videos at the bottom of the tutorial page for extra revisions

Tutorial 01 Notes

- These notes are very similar to that of the notes of the course website, references are the course website.

▼ Overview

Tutorial

- Introduction
- Pointers Revision
- Malloc
- Linked List Revision

Lab (Look at the bottom of the lab for marking)

- Learn how to read and understand sanitizer error messages (Go over in Lab)
- Learn how to use the `make` command (Go over in Lab)
- Get reacquainted with C programming (Go over in Tutorial)
- Practice programming with arrays and linked lists (Go over in Tutorial)

▼ Course Info

Did you receive my welcome email?

Read the Course Outline on the Course Website

▼ Labs

- From Course Website,

- Similar to 1511, each week, there will be a few exercises to work on.
- Completed individually.
- Worth 5 marks each (with an automarking and a handmarking component, I mark you)
 - Look at the bottom of the labs for the marking guideline, you will be marked off that, read this. If you can't find this during the lab ask
- You can have labs marked within two weeks of the lab but prioritise coming in early because you may need this feedback for assignments, you must show on the submission page (double check this before the due date for the lab)
- Made of the best 7 labs (lowest mark disregarded)
 - 2/5, 3/5, 3/5, 1/5, 0/5, 2/5, 3/5, 3/5 so in this case 0/5 is discarded

Remember Lab Assessment

- Bottom of lab it tells you exactly what you need to do for the lab
- You will not get any marks for the lab if you do not attempt the in person hand marking.

▼ Quizzes

- Notes on Quizzes From Course Outline (Please Read Course Outline to Clarify)
 - Released: Beginning of Most Weeks
 - Lecture Content that week and week prior
 - Complete individually on your own time.
 - Marks out of 8
 - Made up of the best 7 quizzes (lowest mark disregarded)

▼ Additional Labs

- Released later in the term but accessible via dropdown on lab page as Week 11-17.

▼ Assignments

- 2 Assignments, more details on course website / outline.

▼ Help sessions

- Available on the course website, will be updated shortly.
- Guide on the bottom of that page to join and find rooms.

▼ Revision Exercises

Extra Revision exercises on the course website

▼ Questions? / Common Questions (OS)

▼ How does hand marking work?

I get you to open up the view submissions / marking page on the course website and click on your submission for the given lab. Then I ask you the questions at the bottom of the lab in the assessments page alongside some additional clarifying questions.

- If it asks you to do something on that page you usually get the marks if you do it. I sometimes ask additional questions but unless they are listed in the assessment page these are just to teach you more about the given exercise.

▼ Course Timetable

- On the website for all the tutorials and labs that are available and the lectures.

▼ Pointers Revision

▼ Use a diagram to represent memory state changes in the lines of this code.

```
int main(void) {  
    int a = 5;  
    int b = 123;  
  
    int *pa = &a;  
    int *pb = &b;  
  
    *pa = 6;  
    *pb = 234;  
  
    int c = *pa;  
}
```

```

    *pa = *pb;
    *pb = c;

    pa = pb;
    *pa = 345;
}

```

▼ Run Program with prints for a b and c

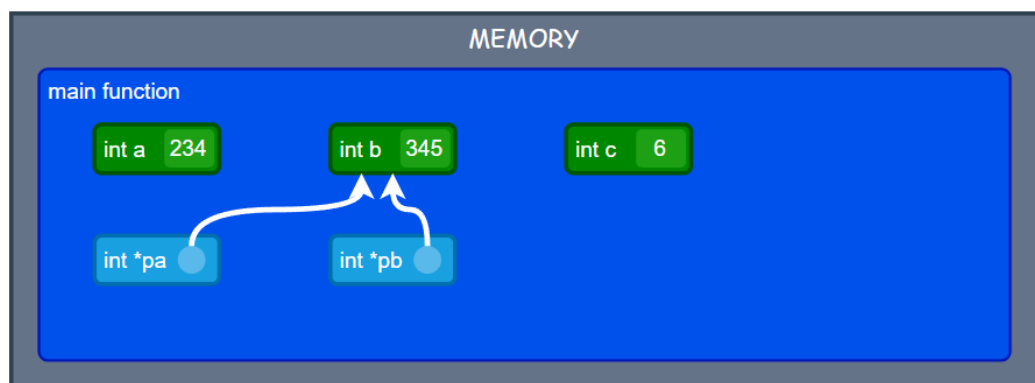
```

clang pointerRev01.c -o pointerRev01
./pointerRev01

```

Clang will be explained during the lab

▼ Final Solution Photo



▼ What makes the swap function not work as intended?

```

int main(void) {
    int a = 5;
    int b = 7;
    swap(&a, &b);
    printf("a = %d, b = %d\n", a, b);
}

void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

```

```
bar() {
    foo()
}

foo(){
```

▼ Answer

The swap function is taking in a and b as copies of the original, this is because besides from arrays C takes in copies of the variables rather than the actual variable.

How can we modify the code to take in what is intended?

```
int main(void) {
    int a = 5;
    int b = 7;
    swap(a, b);
    printf("a = %d, b = %d\n", a, b);
}

void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

Test solution pasting into pointerRev02.c

```
clang pointerRev02.c -o pointerRev02
./pointerRev02
```

▼ Quick Style Tip

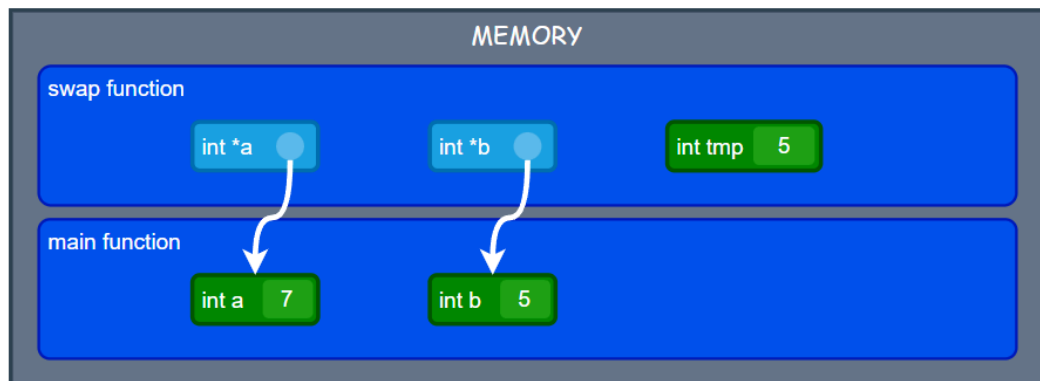
Remember to add prototypes.

▼ Answer

Make a and b, &a and &b, such that they are taking in addresses. This means that the function call for swap would need *a and *b to take in pointers to the addresses to then change.

How does this work using a diagram, step by step through from start of swap function?

▼ Solution Image



▼ Intro to Malloc

▼ What does malloc do? (Answer inside)

Allow the program to allocate memory to the heap. Malloc is a function.

▼ What is the heap (surface level)? (Hinted)

▼ Hint - How is code divided?

Code: Machine code of program located

Data: Global and static variables located (not for this course)

Heap: Dynamically allocate memory located

Stack: Functions and local variables here.

▼ What is the difference between memory allocated to stack and heap?

Stack memory is automatically managed, e.g. cleaning the variables used on return.

Heap memory is allocated `malloc` or `realloc` / `calloc` and deallocated when complete `free`.

▼ When is heap memory useful? (2 Reasons)

When you want to access something later, e.g. you want to access an array that is created in a function outside of the original function.

Very useful when you do not know the size of e.g. how big an array needs to be, because `int arr[100]` and `char b` have fixed size. Do not guess the memory you need for an array by using a large number e.g. `arr[100000]` .

▼ What is the difference between these two pieces of code.

```
// Code 1
int main(void) {
    stackInt();
}

void stackInt(void) {
    int a = 5;
}

// Code 2
int main(void) {
    heapInt();
}

void heapInt(void) {
    int *a = malloc(sizeof(int));
    *a = 5;
}
```

▼ Answer

The `stackInt` allocates only in the stack so when the function runs through returning to `main` (notice without a return call as this is a void function). The `a` variable is then deallocated on return.

In `heapInt` as the variable `a` is `malloc` ed it remains in the heap the only thing that is lost on return is the pointer `*a`. So we cannot access the heap `a`, but it is still there. (Pointers will be removed too on return as they are allocated on the stack and hold the address of the subsequent variable held in the heap, e.g. the `int 5`).

▼ Quick Style Tip

Notice how all function names are camelCase. All variable and function names should be the same casing camelCase or snake_case.

▼ **Use the heap for allocating this struct instead of the stack.**

```
struct node {
    int value;
    struct node *next;
};

int main(void) {
    struct node *n = malloc(sizeof(struct node));
    n->value = 42;
    n->next = NULL;
}
```

▼ **Answer**

Change the main function

```
struct node *n = malloc(sizeof(struct node));
```

or

```
struct node *n = malloc(sizeof(n));
```

then

```
int main(void) {
    struct node *n = malloc(sizeof(n));
    n->value = 42;
    n->next = NULL;
}
```

▼ **Allocate this array on the heap.**

```
int main(void) {
    int *a = malloc(sizeof(int) * 5);
    for (int i = 0; i < 5; i++) {
        a[i] = 42;
    }
}
```



```
}  
}
```

▼ Answer

Only need to change the array to hold $5 * \text{sizeof}(\text{int})$; in a malloc.

▼ Code

```
int main(void) {  
    int *a = malloc(5 * sizeof(int));  
    for (int i = 0; i < 5; i++) {  
        a[i] = 42;  
    }  
}
```

▼ Linked List Revision

▼ Compare the two representations diagrammatically.

```
// Representation 1  
struct node {  
    int value;  
    struct node *next;  
};  
  
int listLength(struct node *list);  
  
// Representation 2  
struct node {  
    int value;  
    struct node *next;  
};  
  
struct list {  
    struct node *head;  
};  
  
int listLength(struct list *list);
```

- Compare the linked lists containing 3, 1 and 4.

▼ Answer

▼ Representation 1

→ [3 | →] [1 | →] [4 | →] [X]

▼ Representation 2

Head

→ [→] [3 | →] [1 | →] [4 | →] [X]

▼ How Should We Represent Empty Lists in These Cases

▼ Answer

▼ Representation 1

→ [X]

▼ Representation 2

Head

→ [→] [X]

▼ What are the advantages of having a separate list struct in representation 2?

▼ Answer

- The list struct can store additional information (metadata) to improve performance (e.g. length, or a pointer to tail and head. But they need to be constantly updated.
- Function that modify the list don't need to return the list, if you have access to the list struct you can access the list after a function is used without needing to update the list struct by setting it equal.

▼ Implement a function to sum the values in the list. Use a while loop, then a for loop

```
struct node {
    int value;
    struct node *next;
};
```

```
// List summing function goes here using while
int sumList(struct node *list) {
    int sum = 0;
    while (list != NULL) {
        sum += list->value;
        list = list->next;
    }
    return sum;
}
```

```
// List summing function goes here using for
int sumList(struct node *list) {
    int sum = 0;
    for (; list != NULL; list = list->next) {
        sum += list->value;
    }
    return sum;
}
```

▼ Answer for while (Hinted)

▼ Hint

▼ Step 1

Create a function that returns an int and takes in a struct node *.

▼ Step 2

Create a sum variable that is equal to 0.

▼ Step 3

Create the while loop from current != NULL adding the elements.

▼ Code

Note: Don't set the sum to equal the first element what if the list has no elements.

This is an edge case that you should consider while coding.

Edge cases are cases that are considered to be unusual cases or unique cases, best given by the examples e.g.

- An empty list
- A list with only one element, with multiple elements

For some questions edge cases can be quite complex.

```
int sumList(struct node *list) {
    struct node *curr = list;
    int sum = 0;
    while (curr != NULL) {
        sum += curr->value;
        curr = curr->next;
    }
    return sum;
}
```

▼ Answer for for

```
int sumList(struct node *list) {
    int sum = 0;
    for (struct node *curr = list; curr != NULL; curr = curr->next) {
        sum += curr->value;
    }
    return sum;
}
```

```
clang linkedList01.c -o linkedList01
./linkedList01
```

▼ Implement a function to delete the first instance of a value from a list. (Hard)

```
struct node {
    int value;
    struct node *next;
};
```

```

struct node *listDelete(struct node *list, int value)
{
    if (list == NULL) {
        return NULL;
    }

    struct node *curr = list;

    if (curr->value == value) {
        struct node *tmp = curr->next;
        free(curr);
        return tmp;
    }

    while (curr != NULL) {
        if (curr->value == value) {
            free(curr);
            return tmp;
        }

        curr = curr->next;
    }

};

```

▼ Hints

▼ Case 1 (Common Check)

```

if (list == NULL) {
    return NULL;
}

```

▼ Case 2

Delete the first value

▼ Case 3

Delete a middle value, otherwise you would have to have a temp pointer follow along.

▼ Answer

```
struct node *listDelete(struct node *list, int value) {
    // Case 1: List is empty
    if (list == NULL) {
        return NULL;

    // Case 2: First element
    } else if (list->value == value) {
        struct node *newHead = list->next;
        free(list);
        return newHead;

    // Case 3: Middle Value
    } else {
        struct node *curr = list;
        while (curr->next != NULL) {
            if (curr->next->value == value) {
                struct node *toDelete = curr->next;
                curr->next = toDelete->next;
                free(toDelete);
                break;
            }
            curr = curr->next;
        }
        return list;
    }
}
```

```
clang linkedList02.c -o linkedList02
./linkedList02
```

How would it be different with this representation.

```
struct node {
    int value;
    struct node *next;
};

struct list {
    struct node *head;
};

struct node *listDelete(struct list *list, int value)
{
    // Case 1: List is empty
    if (list == NULL) {
        return NULL;

    // Case 2: First element
    } else if (list->value == value) {
        struct node *newHead = list->next;
        free(list);
        return newHead;

    // Case 3: Middle Value
    } else {
        struct node *curr = list;
        while (curr->next != NULL) {
            if (curr->next->value == value) {
                struct node *toDelete = curr->next;
                curr->next = toDelete->next;
                free(toDelete);
                break;
            }
            curr = curr->next;
        }
    }
    return list;
}
```

```
}  
}
```

▼ Answer

To access points you would have use `list → head → value` for example and `list → head next`, but you also do not have to return anything as so the prototype can be changed.

- In the first value case remember to reassign `list → head = newHead`.

```
clang linkedList03.c -o linkedList03  
./linkedList03
```

Videos at the bottom of the tutorial page for extra revisions